# Microsoft Sentinel SOAR — Automation Rules & Playbooks (Logic Apps)

Implementation Solution Guide

Version 1.0.0 · 2026-07-09

# Contents


Executive Summary — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps) 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Outcome . . . 6

1. Solution Overview — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps) 6

What this is . . . 6

How the technology works . . . 6

The standard deployment model . . . 6

Naming convention . . . 7

Scope . . . 7

What is out of scope . . . 7

Authentication and connections used . . . 8

Benefits . . . 8

Limitations and honest constraints . . . 8

2. Requirements Guide — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps) 8

2.1 Incidents are being created in Sentinel . . . 8

2.2 Rights to create Azure Logic Apps . . . 9

2.3 Microsoft Sentinel automation permissions . . . 9

2.4 A managed identity strategy for the playbooks . . . 9

2.5 Connector authorizations (Teams, email, ticketing) . . . 10

2.6 Target platforms for response actions (only what you will use) . . . 10

2.7 An approval channel for gated actions . . . 10

2.8 A pilot scope and simulate-first policy defined . . . 11

Checklist . . . 11

Global Requirements — Sentinel Solutions 11

G1. An Azure subscription . . . 12

G2. A Log Analytics workspace . . . 12

G3. Microsoft Sentinel enabled on the workspace . . . 12

G4. Administrative roles (least privilege) . . . 12

G5. Ingestion and retention cost plan . . . 13

G6. Diagnostic settings capability on each source . . . 13

G7. Change control & rollback posture . . . 13

3. Implementation Guide — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps) 13
Step 1 — Confirm Sentinel and prepare the SOAR resource group . . . 14
Step 2 — Build the enrichment playbook (`Enrichment - Incident - Pilot`) . . . 14
Step 3 — Build the notification playbook (`Notify - Teams-Email - Pilot`) . . . 15
Step 4 — Build the ticketing playbook (`Ticketing - Incident - Pilot`) . . . 15
Step 5 — Build the response playbook (`Response - Contain - Pilot`) — approval-gated and simulate-first . . . 15
Step 6 — Secure the playbooks with managed identities (least privilege) . . . 16
Step 7 — Grant Sentinel permission to run the playbooks . . . 16
Step 8 — Create the automation rule (`Auto-Triage - New Incidents - Pilot`) . . . 17
Step 9 — Validate the pilot (the gate) . . . 17
Step 10 — Expand . . . 17
5. Testing Guide — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps) 18
Test 1 — Automation rule fires and triages . . . 18
Test 2 — Enrichment playbook ran . . . 18
Test 3 — Notification delivered . . . 18
Test 4 — Ticket opened and linked . . . 18
Test 5 — Response is approval-gated . . . 18
Test 6 — Response is simulate-first (no real change) . . . 19
Test 7 — Managed identity and least privilege . . . 19
Test 8 — Run history is clean . . . 19
Pass/fail gate . . . 19
6. Troubleshooting Guide — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps) 19
The automation rule runs but the playbook does not . . . 19
Playbook run fails at a Sentinel action ("comment not added / incident not updated") . . . 20
The automation rule never fires . . . 20
Teams or email notification does not arrive . . . 20
The approval card never appears (response playbook) . . . 20
Response playbook made a real change during the pilot . . . 21
Device isolation / user disable action errors when enforced (Expand) . . . 21
Playbook not selectable in the automation rule . . . 21
Costs climbing / too many runs . . . 21
7. Best Practices — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps) 22
Keep response actions simulate-first and approval-gated — always . . . 22
Least-privilege identities, no stored secrets . . . 22

Scope automation rules narrowly, then widen . . . 22
Design playbooks to fail safe and be observable . . . 22
Watch cost and run volume . . . 23
Version and back up your playbooks . . . 23
Where to go next . . . 23
8. Deliverables Summary — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps) 23
Documentation . . . 23
PowerShell scripts . . . 24
What the buyer receives . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

# Executive Summary — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps)

**Sentinel Solutions**

Your analysts do the same manual work on every incident — assign it, tag it, pull the user and IP context, message the on-call, open a ticket, and only then start investigating. That toil is slow, inconsistent, and it burns people out. Microsoft Sentinel already has the machinery to do the routine parts automatically: **automation rules** that fire the moment an incident is created, and **playbooks** (Azure Logic Apps) that enrich, notify, ticket, and — when you decide it is warranted — take contained response actions. This solution builds that layer for you, with a hard rule: nothing disruptive happens without a human approval and a simulate-first dry run.

This is a complete, deploy-it-today package for building a Sentinel SOAR layer, written for an administrator with no prior Logic Apps or automation experience.

## What you get

- A plain-language Solution Overview of the SOAR model — automation rules vs playbooks, how a Logic App is triggered by an incident, and where the approval gates sit.
- A Requirements Guide with verify / implement-if-missing for every prerequisite specific to automation and playbooks.
- An Implementation Guide of precise, click-by-click steps in the required order — build the enrichment playbook, the notify playbook, the ticketing playbook, and the approval-gated response playbook, then wire an automation rule to run them.
- Five PowerShell scripts for SOAR readiness discovery, automation-rule/playbook validation, a coverage and run-history report, a **simulate-only** response dry-run, and a managed-identity/connection security helper.
- Testing and Troubleshooting guides for a confident rollout.

## What this delivers

A working automation layer — an automation rule (`Auto-Triage - New Incidents - Pilot`) that auto-assigns, tags, and triggers playbooks; an enrichment playbook (`Enrichment - Incident - Pilot`) that adds user, IP, and device context to the incident; a notification playbook (`Notify - Teams-Email - Pilot`); a ticketing playbook (`Ticketing - Incident - Pilot`); and an approval-gated response playbook (`Response - Contain - Pilot`) that can disable a user, isolate a device, or block an IP **only after** an explicit approval and only after you have run it in simulate mode — all secured with a managed identity, validated on a pilot before you scale.

## The standard deployment model

`Assess -> Configure -> Pilot -> Validate -> Expand`, with a pilot gate that proves enrichment and notification run cleanly — and that every response action is simulate-first and approval-gated — before any enforced action is switched on.

## Outcome

A SOC that responds at machine speed for the routine work — triage, enrichment, notification, ticketing — while every consequential action (disable, isolate, block) stays behind a human approval and a dry run, so you get speed without surrendering control.

# 1. Solution Overview — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps)

## What this is

A SOAR (Security Orchestration, Automation, and Response) layer for Microsoft Sentinel. It automates the repetitive first minutes of every incident — assignment, tagging, context enrichment, notification, and ticketing — and adds a set of **response actions** (disable a user, isolate a device, block an IP) that are deliberately kept behind an **approval gate** and a **simulate-first** dry run. The goal is machine-speed handling of routine toil with humans in the loop for anything that changes production.

## How the technology works

Microsoft Sentinel gives you two building blocks:

- **Automation rules** — lightweight, no-code rules that run when an incident (or alert) is created or updated. A rule can set severity, assign an owner, add tags, change status, and — most importantly — **run one or more playbooks**. Automation rules are the trigger and the ordering logic; they are evaluated in priority order.
- **Playbooks** — an Azure **Logic App** (Consumption or Standard) that starts from a Sentinel trigger (`Microsoft Sentinel incident` or `Microsoft Sentinel alert`) and runs a sequence of connector actions: query a threat-intel source, look up a user in Entra ID, post an Adaptive Card to Microsoft Teams, send an email, create a ticket, or call Microsoft Defender to isolate a device. Playbooks are where the real work happens.

An incident flows like this: **analytics rule creates an incident → automation rule fires → automation rule assigns/tags the incident and calls the enrichment, notify, and ticketing playbooks → (optionally) an analyst reviews the enriched incident and, if containment is warranted, triggers the response playbook, which pauses for an approval before it does anything disruptive.**

Playbooks authenticate with a **managed identity** (preferred) or an API connection. The managed identity is granted the least-privilege roles the playbook needs (for example, Microsoft Sentinel Responder to change an incident, and the relevant Defender/Graph permission to isolate a device), so there are no stored credentials in the workflow.

## The standard deployment model

This solution follows the same five-stage model as every solution in this program:

```
Assess -> Configure -> Pilot -> Validate -> Expand
```

• **Assess** — confirm Sentinel is onboarded, incidents are being created, and you have rights to create Logic Apps and grant the managed identity its roles.

• **Configure** — build the four playbooks (enrichment, notify, ticketing, response) and the automation rule, all named per the convention below. Keep the response playbook in **simulate mode**.

• **Pilot** — scope the automation rule to a small, safe slice (for example, one analytics rule or one severity) and confirm enrichment, notification, and ticketing run cleanly on real incidents. Run the response playbook only in simulate mode.

• **Validate** — verify the pass/fail gate: enrichment adds context, notifications arrive, tickets open, run history is clean, and every response action is proven simulate-first and approval-gated.

• **Expand** — widen the automation rule scope, add more triggers/severities, and only then — with change approval — enable a real (non-simulated) response action for a specific, well-understood scenario.

## Naming convention

Every object created is named `<Purpose> - <Scope> - <Stage>` and reused verbatim throughout the guide, testing, and scripts:

• Automation rule: `Auto-Triage - New Incidents - Pilot`

• Enrichment playbook (Logic App): `Enrichment - Incident - Pilot`

• Notification playbook: `Notify - Teams-Email - Pilot`

• Ticketing playbook: `Ticketing - Incident - Pilot`

• Response playbook (approval-gated, simulate-first): `Response - Contain - Pilot`

• Managed identity role assignment: `Sentinel Responder - Playbook MI - Pilot`

## Scope

• Sentinel automation rules that auto-assign, tag, and trigger playbooks by analytics rule and/or severity.

• Four Logic App playbooks: incident enrichment, Teams + email notification, ticketing, and approval-gated response.

• Response actions — disable user (Entra ID), isolate device (Microsoft Defender for Endpoint), block IP (a firewall/NSG or threat-intel indicator) — each **simulate-first** and **approval-gated**.

• Managed identity and API-connection security for the playbooks.

• A pilot-first rollout with a safe/simulate mode before any enforced action.

## What is out of scope

• The analytics rules that create the incidents in the first place (covered by the *Sentinel Analytics Rules & MITRE ATT&CK Coverage* solution in this catalog).

• Data ingestion and connectors (covered by *Deploy Microsoft Sentinel — Workspace & Connectors* and *Entra ID & Intune Audit Telemetry to Sentinel*).

• The **decision** of which response actions your organization permits, on whom, and under what authority — that is a security-operations and change-control determination for the customer to own; this guide provides the technical controls, not the policy.

• Third-party SOAR platforms; this solution is native Sentinel + Logic Apps.

### Authentication and connections used

• **Managed identity** on each playbook (system-assigned), granted least-privilege roles: **Microsoft Sentinel Responder** on the workspace/resource group (to update incidents), plus the specific permission each response action needs (for example, the Microsoft Defender for Endpoint machine-isolation permission, or an Entra ID role/Graph permission to disable a user).

• **API connections** for the Teams, Office 365 Outlook, and any ticketing connector, authorized once and reused. Where a managed-identity-based connector is available, prefer it over a stored credential.

### Benefits

• Faster mean-time-to-respond on routine incidents; analysts start on an already-enriched, already-ticketed incident.

• Consistent handling — the same enrichment and notification on every incident, every time.

• Safety by design — nothing disruptive runs without a simulate-first dry run and an explicit approval.

### Limitations and honest constraints

• Logic Apps (Consumption) bill per action/execution; high incident volume has a (modest) cost — size it in the pilot.

• Some connectors (for example, certain ticketing or firewall connectors) are in preview or require a specific license/premium tier — *verify current availability in your tenant* before you depend on one.

• Response actions depend on the target platform being present and licensed (for example, device isolation requires Microsoft Defender for Endpoint onboarded); *verify current availability in your tenant*.

# 2. Requirements Guide — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps)

Verify **every** requirement before configuring. Each entry: **Description / Why / Verify / Implement-if-missing.** Category-wide prerequisites (Azure subscription, Log Analytics workspace, Microsoft Sentinel enabled, RBAC, ingestion/retention cost plan, diagnostic-settings capability, change control) are in `global-requirements-sentinel-solutions.md`; do not repeat them — confirm them there first. The requirements below are specific to automation rules and playbooks.

### 2.1 Incidents are being created in Sentinel

• **Description:** At least one active analytics rule (scheduled, Microsoft-security, or NRT) that produces incidents in the workspace.

• **Why:** Automation rules and playbooks trigger on incidents; with no incidents there is nothing to automate against. The pilot needs a steady, low-volume source of real incidents to test on.

• **Verify:** `portal.azure.com` → `Microsoft Sentinel` → (workspace) → `Incidents` shows recent incidents; `Configuration` → `Analytics` → `Active rules` lists at least one enabled rule. Run `SENTSOAR-Discovery.ps1`.

• **Implement-if-missing:** Enable at least one analytics rule from a template (or deploy the *Sentinel Analytics Rules & MITRE ATT&CK Coverage* solution) so incidents are generated.

## 2.2 Rights to create Azure Logic Apps

• **Description:** Permission to create Logic App (Consumption) resources in the resource group that will host the playbooks.

• **Why:** A playbook is a Logic App; you cannot create one without Contributor (or Logic App Contributor) on the target resource group.

• **Verify:** On the target resource group, `Access control (IAM)` → confirm your account holds Contributor or Logic App Contributor. Try `Create a resource` → `Logic App (Consumption)` and confirm you can reach the create blade.

• **Implement-if-missing:** Have an owner assign Logic App Contributor (or Contributor) on a dedicated resource group for the playbooks (for example `rg-sentinel-soar`), scoped per least privilege.

## 2.3 Microsoft Sentinel automation permissions

• **Description:** The **Microsoft Sentinel Contributor** role to create automation rules, and the **Microsoft Sentinel Playbook Operator** / **Microsoft Sentinel Responder** roles so automation rules can run playbooks and so the playbooks' identity can update incidents.

• **Why:** Creating automation rules, allowing them to run Logic Apps, and letting a playbook modify an incident are each distinct permissions. Sentinel also requires that the **Azure Security Insights** service principal (or the automation rule's permission) be granted rights on the playbook's resource group so a rule may run it.

• **Verify:** On the workspace, `Access control (IAM)` → confirm Sentinel Contributor. In `Configuration` → `Automation` → `Settings`, confirm Sentinel has permission to run playbooks in the SOAR resource group. Run `SENTSOAR-Discovery.ps1`.

• **Implement-if-missing:** Assign Sentinel Contributor on the workspace; in `Automation` → `Active playbooks`, use `Configure permissions` to grant Sentinel run rights on the SOAR resource group.

## 2.4 A managed identity strategy for the playbooks

• **Description:** A decision to use a **system-assigned managed identity** on each playbook (preferred), with least-privilege role assignments, rather than storing credentials in connectors.

• **Why:** A managed identity removes stored secrets from the workflow and lets you grant exactly the roles each playbook needs (Sentinel Responder to update incidents; the specific permission for each response action). This is the secure default and the audit-friendly one.

• **Verify:** Confirm you can enable a system-assigned identity on a Logic App (`Identity` blade) and assign it roles. Run `SENTSOAR-Set-PlaybookIdentity.ps1` to report current identity/role state.

• **Implement-if-missing:** Plan a system-assigned identity per playbook; document the exact roles each needs (see the Implementation Guide) so an owner can assign them at the narrowest scope.

## 2.5 Connector authorizations (Teams, email, ticketing)

• **Description:** The ability to authorize the API connections the notification and ticketing playbooks use — Microsoft Teams, Office 365 Outlook (email), and your ticketing system's connector (for example ServiceNow, Jira, or a generic HTTP/webhook to an ITSM).

• **Why:** Each connector needs a one-time authorization (an account or a managed identity) before a playbook can post to Teams, send email, or open a ticket.

• **Why version/license matters:** some ticketing connectors are premium or in preview and may require a specific plan — *verify current availability in your tenant*.

• **Verify:** In `Logic Apps` → an existing or scratch playbook → add a Teams / Outlook action and confirm the connection authorizes. Confirm which ticketing connector your organization uses and that an account exists to authorize it.

• **Implement-if-missing:** Provision a low-privilege service account (or managed identity) to own the Teams/Outlook/ticketing connections; authorize each once. If no ITSM connector is available, use an HTTP action to the ITSM's inbound webhook/API.

## 2.6 Target platforms for response actions (only what you will use)

• **Description:** The platforms that the response actions act on must be present and licensed: **Entra ID** to disable a user; **Microsoft Defender for Endpoint** onboarded to isolate a device; a **firewall/NSG or Sentinel threat-intel indicator** path to block an IP.

• **Why:** A response action cannot run against a platform you do not have. You only need the ones you intend to use — the response playbook is modular.

• **Why license matters:** device isolation requires Microsoft Defender for Endpoint (P1/P2 as applicable); some block-IP paths require a specific firewall connector or premium tier — *verify current availability in your tenant*.

• **Verify:** For device isolation: `security.microsoft.com` → devices are onboarded. For user-disable: an Entra ID role/Graph permission that can disable an account. For block-IP: confirm the firewall connector or the threat-intel indicators path is available. Run `SENTSOAR-Validate.ps1`.

• **Implement-if-missing:** Only implement the platforms for the response actions you will pilot; leave the rest out of scope until needed.

## 2.7 An approval channel for gated actions

• **Description:** A Microsoft Teams channel or an email distribution list that receives the approval prompt before any disruptive response action runs, with the named approvers who may approve.

• **Why:** The response playbook pauses and posts an approval request; someone with authority must approve before it proceeds. This is the human-in-the-loop control.

• **Verify:** Confirm a Teams channel or shared mailbox exists and that the approvers are identified and agree to the process.

• **Implement-if-missing:** Create a dedicated `SOC Approvals` Teams channel (or mailbox); document who may approve which action.

### 2.8 A pilot scope and simulate-first policy defined

• **Description:** A named, limited pilot scope (for example, incidents from one analytics rule or one severity) with success criteria, and a written policy that **all response actions run in simulate mode first** and never enforce during the pilot.

• **Why:** The standard model validates the automation on a safe slice, and containment must be proven harmless (simulate) before it is ever allowed to enforce.

• **Verify:** The pilot scope, pass criteria, and simulate-first policy are written down. Run `SENTSOAR-Report.ps1` to confirm run history and that no enforced response ran during the pilot.

• **Implement-if-missing:** Choose the smallest useful incident slice as the pilot; record the gate (enrichment + notify + ticketing clean, response simulate-only) before expanding.

### Checklist

• Azure subscription, Log Analytics workspace, and Microsoft Sentinel enabled — see global requirements

• Incidents are being created by at least one analytics rule — `SENTSOAR-Discovery.ps1`

• Rights to create Logic Apps in a dedicated SOAR resource group

• Sentinel automation/playbook permissions in place (rule can run playbooks) — `SENTSOAR-Discovery.ps1`

• Managed-identity strategy with least-privilege roles — `SENTSOAR-Set-PlaybookIdentity.ps1`

• Teams / email / ticketing connectors can be authorized

• Target platforms present for the response actions you will use — `SENTSOAR-Validate.ps1`

• Approval channel and named approvers agreed

• Pilot scope and simulate-first policy defined — `SENTSOAR-Report.ps1`

## Global Requirements — Sentinel Solutions

These requirements are common to **every** Sentinel Solutions solution in the standard program model. They are the baseline Azure Monitor / Microsoft Sentinel platform prerequisites that any telemetry, detection, or SIEM engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

> *Naming convention: every created object is named* `<Purpose> - <Scope> - <Stage>` *and reused verbatim throughout the engagement (for example* `Audit Telemetry - Tenant - Pilot`*).*

## G1. An Azure subscription

• **Description:** An active Azure subscription in the tenant in scope, with billing enabled.

• **Why:** A Log Analytics workspace and Microsoft Sentinel are Azure resources billed against a subscription; nothing can be provisioned without one.

• **Verify:** Sign in to `https://portal.azure.com`, open `Subscriptions`, and confirm an active subscription resolves and shows status `Active`.

• **Implement-if-missing:** Create or associate an Azure subscription (Enterprise Agreement, CSP, or pay-as-you-go) and confirm billing is in place.

## G2. A Log Analytics workspace

• **Description:** A Log Analytics workspace to receive and store ingested telemetry, sized to the data volume and retention in scope.

• **Why:** Microsoft Sentinel runs on top of a Log Analytics workspace; all log data lands here and all queries/analytics run against it.

• **Why region matters:** the workspace region determines data residency and affects egress; pick one deliberately before ingestion starts (it cannot be moved later).

• **Verify:** `portal.azure.com` → `Log Analytics workspaces` → confirm the target workspace exists, note its region and resource group.

• **Implement-if-missing:** Create a workspace named `Security Telemetry - Tenant - Workspace`; choose the region for residency; select the pricing tier that fits expected daily ingestion.

## G3. Microsoft Sentinel enabled on the workspace

• **Description:** Microsoft Sentinel added to (enabled on) the Log Analytics workspace in scope.

• **Why:** Connectors, analytics rules, hunting, and incidents are Sentinel features layered on the workspace; without Sentinel enabled you have logs but no SIEM.

• **Verify:** `portal.azure.com` → `Microsoft Sentinel` → confirm the workspace is listed.

• **Implement-if-missing:** `Microsoft Sentinel` → `Create` → select the workspace → `Add`.

## G4. Administrative roles (least privilege)

• **Description:** Appropriately scoped roles for the work — Microsoft Sentinel Contributor (or Log Analytics Contributor) on the workspace for configuration, plus the tenant roles needed to set diagnostic settings on the sources (for example a role that can edit Entra ID diagnostic settings).

• **Why:** Each configuration action requires sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** On the workspace/resource group, `Access control (IAM)` → confirm the operating accounts hold the required roles. In Entra → `Roles & admins`, confirm the role that can edit diagnostic settings.

• **Implement-if-missing:** Assign the minimum roles needed at the resource-group or workspace scope; prefer eligible (PIM) activation over standing access where available.

### G5. Ingestion and retention cost plan

• **Description:** A documented estimate of daily ingestion (GB/day) and a retention decision (interactive retention vs long-term/archive), with a budget owner.

• **Why:** Sentinel and Log Analytics are billed primarily on data ingested and data retained; identity and device audit logs can be high-volume, so cost must be planned before connectors are switched on.

• **Verify:** Confirm a written estimate exists (sources × expected GB/day) and that retention periods are agreed per table or workspace.

• **Implement-if-missing:** Estimate volume from the source log categories in scope; choose pay-as-you-go or a commitment tier; set workspace/table retention; record the budget and owner.

### G6. Diagnostic settings capability on each source

• **Description:** The ability to create diagnostic settings (or enable the relevant data connector) on every source whose telemetry is in scope.

• **Why:** Telemetry only flows to the workspace when a diagnostic setting (or connector) routes the source's log categories to it; this is the switch that turns the pipeline on.

• **Verify:** For each source, confirm a diagnostic setting can be created and that no existing setting already routes the same categories elsewhere (to avoid duplicate ingestion and double billing).

• **Implement-if-missing:** Plan one named diagnostic setting per source that forwards the in-scope categories to the workspace (named per G's convention).

### G7. Change control & rollback posture

• **Description:** An approved change process with a scheduled window, plus a documented rollback (which diagnostic settings/connectors/analytics rules were added) before any change.

• **Why:** Enabling connectors and analytics rules changes ingestion volume (and therefore cost) and can generate incidents; controlled changes and a recorded back-out keep the pipeline safe and auditable.

• **Verify:** Confirm an approved change record and a written list of objects to be created (so they can be removed cleanly).

• **Implement-if-missing:** Stand up a lightweight change/approval process; capture the pre-change configuration; document the rollback (delete the named diagnostic setting / disable the connector / disable the rule).

## 3. Implementation Guide — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps)

Follow these steps **in order**. They use the Azure portal (`https://portal.azure.com`) for Microsoft Sentinel, Logic Apps, and role assignments. Build the playbooks first, secure them with a managed

identity, keep the response playbook in **simulate mode**, then wire an automation rule to run them on a small pilot slice before expanding.

> *Model: build enrichment → notify → ticketing → response (simulate-first, approval-gated) → automation rule → validate on a pilot (the gate) → expand. Every object created is named* `<Purpose> - <Scope> - <Stage>` *and reused verbatim.*

Objects this guide creates (name them exactly):

- Resource group (host for playbooks): `rg-sentinel-soar`
- Enrichment playbook (Logic App): `Enrichment - Incident - Pilot`
- Notification playbook: `Notify - Teams-Email - Pilot`
- Ticketing playbook: `Ticketing - Incident - Pilot`
- Response playbook (approval-gated, simulate-first): `Response - Contain - Pilot`
- Automation rule: `Auto-Triage - New Incidents - Pilot`
- Managed-identity role assignment: `Sentinel Responder - Playbook MI - Pilot`

## Step 1 — Confirm Sentinel and prepare the SOAR resource group

1. `portal.azure.com` → search `Microsoft Sentinel` → confirm your workspace is listed and `Incidents` shows recent incidents. If none exist, enable an analytics rule first (Requirement 2.1).

2. `portal.azure.com` → `Resource groups` → `Create` → name it `rg-sentinel-soar`, same region as the workspace → `Review + create` → `Create`. This keeps all playbooks in one place for permissions and cost tracking.

3. Run `SENTSOAR-Discovery.ps1` to confirm Sentinel onboarding, existing automation rules/playbooks, and your permissions.

## Step 2 — Build the enrichment playbook (`Enrichment - Incident - Pilot`)

1. `Create a resource` → search `Logic App` → `Logic App (Consumption)` → `Create`. Resource group `rg-sentinel-soar`; name `Enrichment - Incident - Pilot`; same region → `Review + create` → `Create`.

2. Open it → `Logic app designer` → start with a blank Logic App → add trigger `Microsoft Sentinel incident` (`When a Microsoft Sentinel incident creation rule is triggered`).

3. Add action `Microsoft Sentinel` → `Entities - Get Accounts` and `Entities - Get IPs` to pull the user and IP entities from the incident.

4. Add enrichment lookups (use only the ones you have): for each IP, a **threat-intel** or **VirusTotal** connector action (or an `HTTP` GET to your TI source); for each account, an **Entra ID (`HTTP` to Microsoft Graph or the Azure AD connector)** lookup for account status/risk.

5. Add `Microsoft Sentinel` → `Add comment to incident` and post the collected context (IP reputation, account risk/status) back onto the incident as a formatted comment.

6. Save. In Step 6 you will enable its managed identity and grant it Sentinel Responder.

## Step 3 — Build the notification playbook (Notify - Teams-Email - Pilot)

1. Create another Logic App (Consumption) in rg-sentinel-soar named Notify - Teams-Email - Pilot.

2. Designer → trigger Microsoft Sentinel incident.

3. Add Microsoft Teams → Post adaptive card in a chat or channel targeting the SOC Approvals/SOC channel (Requirement 2.7). Build a card with the incident title, severity, owner, and a deep link (Microsoft Sentinel → dynamic Incident URL).

4. Add Office 365 Outlook → Send an email (V2) to the SOC distribution list with the same summary (belt-and-braces so a missed Teams post is still emailed).

5. Save. Authorize the Teams and Outlook connections with the low-privilege service account (Requirement 2.5).

## Step 4 — Build the ticketing playbook (Ticketing - Incident - Pilot)

1. Create a Logic App (Consumption) in rg-sentinel-soar named Ticketing - Incident - Pilot.

2. Designer → trigger Microsoft Sentinel incident.

3. Add your ITSM action: ServiceNow → Create Record, Jira → Create issue, or an HTTP **POST** to your ITSM's inbound API/webhook if no first-party connector exists. Map incident title → summary, severity → priority, incident URL → description.

4. Add Microsoft Sentinel → Add comment to incident writing the ticket number/URL back onto the incident so the two systems are linked.

5. Save. Authorize the ticketing connection (Requirement 2.5).

## Step 5 — Build the response playbook (Response - Contain - Pilot) — approval-gated and simulate-first

1. Create a Logic App (Consumption) in rg-sentinel-soar named Response - Contain - Pilot.

2. Designer → trigger Microsoft Sentinel incident (this playbook is intended to be run **manually** by an analyst from the incident, or by the automation rule only in Expand).

3. Add a parameter/variable SimulateMode (boolean) initialized to true. Every action below is wrapped in a condition on SimulateMode.

4. Add Microsoft Teams → Post an Adaptive Card and wait for a response (an approval card) to the SOC Approvals channel: "Approve containment for incident *[title]* — action: *[disable user / isolate device / block IP]*?" This is the human-in-the-loop **approval gate**; the playbook pauses here until an approver responds.

5. Add a Condition: proceed only if the approval response is Approve.

6. Inside the approved branch, add a nested `Condition` **on** `SimulateMode`:

• **If** `SimulateMode` **is** `true` **(default):** do **not** call any platform. Instead `Add comment to incident`: "SIMULATE: would disable user *[UPN]* / isolate device *[id]* / block IP *[ip]* — no change made." This is the dry run.

• **If** `SimulateMode` **is** `false` **(enforced, Expand only):** call the real action for the specific containment you are enabling:

• **Disable user** — `Azure AD/HTTP to Microsoft Graph` `PATCH /users/{id} accountEnabled=false`, or the `Microsoft Entra ID` connector's disable action.

• **Isolate device** — `Microsoft Defender for Endpoint` → `Isolate machine` (requires Defender for Endpoint onboarded — Requirement 2.6).

• **Block IP** — your firewall/NSG connector action, or `Microsoft Sentinel` → add a threat-intel indicator to block on downstream, per your architecture.

1. After the action (real or simulated), `Add comment to incident` recording what was done, by whom (the approver), and when. Optionally `Change incident status` to `Active`/`Closed` per your process.

2. `Save`. **Leave** `SimulateMode = true` **for the entire pilot.** It stays simulate-first until an explicit change approval flips it for one specific action in Expand.

## Step 6 — Secure the playbooks with managed identities (least privilege)

1. For **each** playbook: open it → `Identity` → `System assigned` → `On` → `Save`. Note the object (principal) ID.

2. Grant each identity only what it needs, at the narrowest scope:

• **All four playbooks** need to update incidents: on the workspace/resource group, `Access control (IAM)` → `Add role assignment` → **Microsoft Sentinel Responder** → assign to the playbook's managed identity. Record this assignment as `Sentinel Responder - Playbook MI - Pilot`.

• `Response - Contain - Pilot` additionally needs the permission for the *enforced* action you will eventually enable — e.g. the Microsoft Defender for Endpoint machine-isolation permission, or a Graph/Entra role that can disable a user. **Grant these only when you move that action to enforced in Expand**, not during the simulate-only pilot.

1. Run `SENTSOAR-Set-PlaybookIdentity.ps1` to report each playbook's identity and its role assignments and confirm least privilege.

## Step 7 — Grant Sentinel permission to run the playbooks

1. `Microsoft Sentinel` → (workspace) → `Configuration` → `Automation` → `Active playbooks` (or `Settings`).

2. Use `Configure permissions` → select `rg-sentinel-soar` → `Apply`. This grants the Microsoft Sentinel service the rights to run playbooks in that resource group from automation rules (Requirement 2.3).

## Step 8 — Create the automation rule (`Auto-Triage - New Incidents - Pilot`)

1. `Microsoft Sentinel` → `Configuration` → `Automation` → `Create` → `Automation rule`.

2. Name: `Auto-Triage - New Incidents - Pilot`.

3. **Trigger:** `When incident is created`.

4. **Conditions (scope the pilot narrowly):** add a condition such as `Analytics rule name` `Contains` your one pilot rule, **or** `Severity` `Equals` `Medium` — a small, safe slice (Requirement 2.8).

5. **Actions (in order):**

• `Assign owner` → the pilot analyst or SOC group.

• `Add tags` → `soar-pilot`, `auto-triaged`.

• `Run playbook` → `Enrichment - Incident - Pilot`.

• `Run playbook` → `Notify - Teams-Email - Pilot`.

• `Run playbook` → `Ticketing - Incident - Pilot`.

• **Do not** add `Response - Contain - Pilot` here during the pilot — it stays analyst-triggered and simulate-only.

1. Set `Order` (priority) and `Save`.

## Step 9 — Validate the pilot (the gate)

1. Run `SENTSOAR-Validate.ps1` → confirms the automation rule is enabled, each playbook is enabled with a managed identity, and Sentinel has run permission on `rg-sentinel-soar`.

2. Trigger a real (or test) incident in the pilot scope. Confirm within a few minutes:

• The incident is auto-assigned and tagged (`soar-pilot`, `auto-triaged`).

• `Enrichment - Incident - Pilot` added an enrichment comment.

• `Notify - Teams-Email - Pilot` posted to Teams and emailed the SOC.

• `Ticketing - Incident - Pilot` opened a ticket and wrote the ticket number back.

1. Manually run `Response - Contain - Pilot` from the incident (`Actions` → `Run playbook`). Approve the card. Confirm it posted a **SIMULATE** comment and made **no** platform change (`SimulateMode = true`).

2. Run `SENTSOAR-Report.ps1` → review run history, success/failure counts, and confirm **no enforced response action** ran during the pilot.

3. Complete the Testing Guide. **Pilot gate:** do not expand until enrichment/notify/ticketing run cleanly and the response playbook is proven approval-gated and simulate-only.

## Step 10 — Expand

1. Once the gate passes, widen `Auto-Triage - New Incidents - Pilot` (or clone it to `Auto-Triage - New Incidents - Production`) to cover more analytics rules/severities.

2. If — and only if — your change-control process approves enforcing a specific containment for a specific scenario: grant `Response - Contain - Pilot`'s managed identity the exact platform permission (Step 6.2), set that action's branch to `SimulateMode = false` for that one action, keep the approval gate, and test on a single controlled case first. Rename to `Response - Contain - Production`.

3. Re-run `SENTSOAR-Report.ps1` each cycle to track run volume, success rate, and Logic App cost.

### *Order of operations*

```
Sentinel + SOAR resource group -> enrichment playbook -> notify playbook ->
ticketing playbook -> response playbook (simulate-first, approval-gated) ->
managed identities + least-privilege roles -> Sentinel run permission ->
automation rule (narrow pilot) -> validate on a pilot (gate) -> expand
```

# 5. Testing Guide — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps)

Run after the pilot is configured (Implementation Step 9) and after each expansion. Use a real incident in the pilot scope, or create a controlled test incident.

## Test 1 — Automation rule fires and triages

**Do:** Trigger (or wait for) an incident in the pilot scope. `Microsoft Sentinel` → `Incidents` → open it. Or run `SENTSOAR-Validate.ps1`. **Expected:** The incident is auto-assigned to the pilot owner and carries the `soar-pilot` and `auto-triaged` tags applied by `Auto-Triage - New Incidents - Pilot`.

## Test 2 — Enrichment playbook ran

**Do:** On the same incident, read the `Comments`/`Activity log`. Or `Logic Apps` → `Enrichment - Incident - Pilot` → `Runs history`. **Expected:** An enrichment comment is present (IP reputation and/or account status/risk), and the latest run shows `Succeeded`.

## Test 3 — Notification delivered

**Do:** Check the SOC Teams channel and the SOC mailbox. Or `Notify - Teams-Email - Pilot` → `Runs history`. **Expected:** An Adaptive Card posted to Teams with the incident summary and deep link, and an email arrived; the run shows `Succeeded`.

## Test 4 — Ticket opened and linked

**Do:** Check the ITSM for a new ticket; read the incident comment for the ticket number/URL. Or `Ticketing - Incident - Pilot` → `Runs history`. **Expected:** A ticket exists with the incident details, and the ticket number is written back onto the incident.

## Test 5 — Response is approval-gated

**Do:** On an incident, `Actions` → `Run playbook` → `Response - Contain - Pilot`. Watch the SOC Approvals channel. **Expected:** The playbook pauses and posts an approval card; nothing proceeds until an approver clicks `Approve`. If `Reject` is chosen (or it times out), no action is taken.

### Test 6 — Response is simulate-first (no real change)

**Do:** Approve the card from Test 5 while `SimulateMode = true` (pilot default). Then verify the target was **not** changed — the user is still enabled / the device is not isolated / the IP is not blocked. **Expected:** The incident gets a `SIMULATE: would ...` comment, and there is **no** change on any platform. `Response - Contain - Pilot` → `Runs history` shows it took the simulate branch.

### Test 7 — Managed identity and least privilege

**Do:** Run `SENTSOAR-Set-PlaybookIdentity.ps1`. **Expected:** Each playbook has a system-assigned identity; each holds Microsoft Sentinel Responder and no more than it needs. The response playbook has no enforced-action platform permission during the pilot.

### Test 8 — Run history is clean

**Do:** Run `SENTSOAR-Report.ps1`. **Expected:** A per-playbook and automation-rule summary of runs, success/failure counts, and a confirmation that **no enforced (non-simulate) response action** ran during the pilot.

### Pass/fail gate

| Gate | Requirement |
|---|---|
| Pilot pass | Tests 1-8 pass: triage, enrichment, notification, and ticketing run cleanly on real incidents; the response playbook is proven approval-gated and simulate-only; identities are least-privilege |
| Expand pass | After change approval, a single enforced response action runs correctly against one controlled case with the approval gate intact, and cost/run-volume is within budget, before wider rollout |

## 6. Troubleshooting Guide — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps)

Common failures, their cause, and the fix. Work top-down; most issues are permissions or scope.

### The automation rule runs but the playbook does not

- **Symptom:** The incident is assigned/tagged, but no enrichment/notify/ticket comment appears; the playbook shows no run.

• **Cause:** Microsoft Sentinel does not have permission to run playbooks in the SOAR resource group.

• **Fix:** `Microsoft Sentinel` → `Configuration` → `Automation` → `Active playbooks` → `Configure permissions` → select `rg-sentinel-soar` → `Apply` (Implementation Step 7). Re-trigger the incident and check the playbook `Runs history`.

## Playbook run fails at a Sentinel action ("comment not added / incident not updated")

• **Symptom:** The Logic App run shows `Failed` on `Add comment to incident` or `Change incident status`, often with a 403.

• **Cause:** The playbook's managed identity lacks **Microsoft Sentinel Responder** on the workspace/resource group.

• **Fix:** Enable the system-assigned identity (`Identity` blade), then assign Microsoft Sentinel Responder to it (`SENTSOAR-Set-PlaybookIdentity.ps1` or Implementation Step 6). Re-run the failed run.

## The automation rule never fires

• **Symptom:** New incidents get no auto-triage at all.

• **Cause:** The rule's conditions do not match (too narrow), the rule is disabled, or another higher-priority rule changed the incident first.

• **Fix:** `Automation` → open `Auto-Triage - New Incidents - Pilot` → confirm it is `Enabled` and its condition (analytics rule name / severity) actually matches your pilot incidents. Check `Order` so it is not pre-empted. Widen the condition if needed, but keep the pilot narrow.

## Teams or email notification does not arrive

• **Symptom:** Enrichment works but no Teams card / no email.

• **Cause:** The Teams or Office 365 Outlook API connection is unauthorized, expired, or the target channel/mailbox is wrong.

• **Fix:** Open `Notify - Teams-Email - Pilot` → the Teams/Outlook action → re-authorize the connection with the low-privilege service account (Requirement 2.5). Confirm the channel ID / recipient. Re-run.

## The approval card never appears (response playbook)

• **Symptom:** `Response - Contain - Pilot` starts but the SOC Approvals channel gets no card, or it never resumes.

• **Cause:** The Teams connection for the approval action is unauthorized, the channel is wrong, or the approvers cannot see the channel.

• **Fix:** Re-authorize the Teams connection; confirm the `SOC Approvals` channel and that approvers are members (Requirement 2.7). Note that `Post an Adaptive Card and wait for a response` will wait up to the Logic App run limit — a very old run may have timed out; re-run.

## Response playbook made a real change during the pilot

- **Symptom:** A user was actually disabled / a device actually isolated during the pilot.
- **Cause:** `SimulateMode` was set to `false`, or the enforced branch was not correctly gated behind the `SimulateMode` condition.
- **Fix:** This must not happen in the pilot. Open `Response - Contain - Pilot` → confirm the `SimulateMode` variable initializes to `true` and that every platform action sits inside the `SimulateMode = false` branch only (Implementation Step 5). Remove any enforced-action role from the playbook's identity for the pilot (`SENTSOAR-Set-PlaybookIdentity.ps1` flags this). Reverse the change on the affected user/device and record it.

## Device isolation / user disable action errors when enforced (Expand)

- **Symptom:** After moving an action to enforced, it fails with a permission or "not found" error.
- **Cause:** The managed identity lacks the specific platform permission (Defender for Endpoint isolation permission, or a Graph/Entra role to disable a user), or the target platform is not onboarded/licensed.
- **Fix:** Grant the identity the exact least-privilege permission for that one action (Implementation Step 6.2 / Step 10). Confirm Defender for Endpoint is onboarded and the device/user resolves (`SENTSOAR-SimulateResponse.ps1` verifies the target). *Verify current availability/licensing in your tenant.*

## Playbook not selectable in the automation rule

- **Symptom:** `Run playbook` in the automation rule does not list your playbook.
- **Cause:** The playbook does not use the `Microsoft Sentinel incident` (or alert) trigger, or Sentinel lacks run permission on its resource group.
- **Fix:** Confirm each playbook starts from the `Microsoft Sentinel incident` trigger (Implementation Steps 2-5) and that `Configure permissions` covered `rg-sentinel-soar` (Step 7).

## Costs climbing / too many runs

- **Symptom:** Logic App action count and cost rise faster than expected.
- **Cause:** The automation rule scope is too broad, or a playbook has many actions running on every incident.
- **Fix:** Run `SENTSOAR-Report.ps1` to see run counts per playbook. Narrow the automation-rule condition, split heavy enrichment into a separate playbook, or reduce per-run actions. Add an expiration to pilot rules.

If a problem persists, re-run `SENTSOAR-Discovery.ps1` and `SENTSOAR-Validate.ps1` and compare against the objects named in the Implementation Guide — nearly every issue is a missing role, an unauthorized connection, or a mis-scoped automation rule.

# 7. Best Practices — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps)

Field-proven guidance for keeping the SOAR layer safe, cheap, and maintainable.

## Keep response actions simulate-first and approval-gated — always

- Every disruptive action (disable user, isolate device, block IP) stays behind **both** a human approval **and** a `SimulateMode` dry run. Move an action to enforced only for a specific, well-understood scenario, one at a time, with change approval.
- Never wire the response playbook into an automation rule that runs on incident creation during the pilot. Containment stays analyst-triggered until you have proven the scenario.
- Grant the response playbook's managed identity the enforced-action permission **only when** you enable that action — not before. A simulate-only playbook needs no containment rights.

## Least-privilege identities, no stored secrets

- Prefer a **system-assigned managed identity** on every playbook over stored connector credentials. Grant only Microsoft Sentinel Responder for incident updates, plus the single permission each enforced action needs.
- Scope role assignments at the resource group or workspace, not the subscription. Review them each quarter and remove anything unused.
- For connectors that still need an account (some Teams/Outlook/ITSM connectors), use a dedicated low-privilege service account, not a person's credentials.

## Scope automation rules narrowly, then widen

- Start with one analytics rule or one severity. A too-broad rule that runs playbooks on every incident is both a cost and a noise risk.
- Use automation-rule **order/priority** deliberately so triage (assign/tag) happens before playbooks, and so rules do not conflict.
- Add an expiration date to pilot automation rules so they cannot outlive the pilot unnoticed.

## Design playbooks to fail safe and be observable

- Put enrichment and notification in **separate** playbooks so one failing connector does not block the others.
- Add error handling (`Configure run after`) so a failed enrichment still posts a "enrichment unavailable" comment rather than silently dying.
- Always write an incident comment recording what each playbook did — enrichment, ticket number, notification sent, simulate/enforce decision, and (for response) the approver. The incident becomes its own audit trail.

### Watch cost and run volume

• Logic Apps (Consumption) bill per action execution; a chatty enrichment playbook on a high-volume rule adds up. Review `SENTSOAR-Report.ps1` run counts and trim actions or narrow scope.

• Prefer a few well-built playbooks over many overlapping ones — fewer runs, less to maintain.

### Version and back up your playbooks

• Export each Logic App's ARM/template (or keep it in source control) so a playbook can be rebuilt or promoted (Pilot → Production) cleanly.

• Rename on promotion (`... - Pilot` → `... - Production`) and keep the pilot copy until the production one is proven.

### Where to go next

• **Sentinel Analytics Rules & MITRE ATT&CK Coverage** — build the detections that create the incidents this SOAR layer acts on, and map coverage to ATT&CK.

• **SOC Onboarding — Sentinel & Defender XDR** — unify incidents across Sentinel and Defender so your playbooks act on a single queue.

• **Sentinel Cost & Data-Tier Optimization** — keep the ingestion feeding these detections lean so the whole SIEM/SOAR stack stays affordable.

• **Deploy Microsoft Sentinel — Workspace & Connectors** — if you still need to stand up or extend the underlying data sources.

## 8. Deliverables Summary — Microsoft Sentinel SOAR: Automation Rules & Playbooks (Logic Apps)

### Documentation

| Document | Purpose |
|---|---|
| Executive Summary | Business case and what's in the box |
| 1. Solution Overview | The SOAR model — automation rules vs playbooks, how a Logic App is triggered by an incident, the approval gates, scope, benefits, limitations |
| 2. Requirements Guide | Every automation/playbook prerequisite with Verify / Implement-if-missing |
| Global Requirements (Sentinel Solutions) | Category-wide prerequisites, published once |
| 3. Implementation Guide | Exact, ordered steps — build enrichment, notify, ticketing, and approval-gated response playbooks; managed identities; the automation rule |

| Document | Purpose |
|---|---|
| 5. Testing Guide | Validation procedures with pass/fail gates (including approval-gated and simulate-first proofs) |
| 6. Troubleshooting Guide | Field issues and remediation |
| 7. Best Practices | Field-proven recommendations |

## PowerShell scripts

| Script | What it does |
|---|---|
| `SENTSOAR-Discovery.ps1` | Reports SOAR readiness — Sentinel onboarding, existing automation rules and playbooks, incidents present, and your permissions (read-only) |
| `SENTSOAR-Validate.ps1` | Confirms the automation rule is enabled, each playbook is enabled with a managed identity, and target platforms for response actions are reachable (read-only) |
| `SENTSOAR-Report.ps1` | Playbook and automation-rule run-history report — success/failure counts, cost signal, and a check that no enforced response ran during the pilot (read-only) |
| `SENTSOAR-SimulateResponse.ps1` | Simulate-only dry run of the containment logic — prints what a disable-user / isolate-device / block-IP would target, makes no change (read-only, safe) |
| `SENTSOAR-Set-PlaybookIdentity.ps1` | Managed-identity and connection security helper — reports each playbook's identity and roles; can assign least-privilege Sentinel Responder (default `-WhatIf`) |

## What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the standard model: `Assess -> Configure -> Pilot -> Validate -> Expand`.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

## Outcome

A native Sentinel SOAR layer — automation rules that triage, playbooks that enrich, notify, and ticket, and a containment playbook that stays approval-gated and simulate-first — validated on a pilot before any enforced action is switched on.

# Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

## SENTSOAR-Discovery.ps1

**Purpose:** Reports whether the SOAR building blocks are in place: Microsoft Sentinel onboarded on the workspace, incidents being created, existing automation rules and playbooks (Logic Apps), and whether Sentinel has run permission on the SOAR resource group. Read-only — no changes. HOW TO RUN Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Discovery.ps1 -ResourceGroup <workspace-rg> -WorkspaceName "Security Telemetry - Tenant - Workspace" -SoarResourceGroup rg-sentinel-soar Requires modules: Az.Accounts, Az.OperationalInsights, Az.SecurityInsights, Az.LogicApp, Az.Resources. EXPECTED RESULTS Sentinel enabled (yes/no), recent-incident count, list of automation rules and playbooks, and a note on whether the SOAR resource group hosts Logic Apps. Highlights anything missing. .NOTES Sentinel Solutions. Read-only. > [CmdletBinding()] param( [Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [string]$SoarResourceGroup = 'rg-sentinel-soar' )

**How to run:** Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Discovery.ps1 -ResourceGroup <workspace-rg> -WorkspaceName "Security Telemetry - Tenant - Workspace" -SoarResourceGroup rg-sentinel-soar Requires modules: Az.Accounts, Az.OperationalInsights, Az.SecurityInsights, Az.LogicApp, Az.Resources. EXPECTED RESULTS Sentinel enabled (yes/no), recent-incident count, list of automation rules and playbooks, and a note on whether the SOAR resource group hosts Logic Apps. Highlights anything missing. .NOTES Sentinel Solutions. Read-only. > [CmdletBinding()] param( [Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [string]$SoarResourceGroup = 'rg-sentinel-soar' )

## SENTSOAR-Report.ps1

**Purpose:** Summarizes recent Logic App run history for each playbook (succeeded/failed/running counts) as a cost and reliability signal, and flags any run of the response playbook so you can confirm no ENFORCED (non-simulate) containment ran during the pilot. Read-only — no changes. HOW TO RUN Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Report.ps1 -SoarResourceGroup rg-sentinel-soar -Days 7 Requires modules: Az.Accounts, Az.LogicApp. EXPECTED RESULTS A per-playbook table of run counts by status over the window, and a highlighted list of any 'Response - Contain - Pilot' runs for manual review. Use run counts as a Logic App cost proxy. .NOTES Sentinel Solutions. Read-only. > [CmdletBinding()] param( [string]$SoarResourceGroup = 'rg-sentinel-soar', [int]$Days = 7, [string]$ResponsePlaybook = 'Response - Contain - Pilot' )

**How to run:** Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Report.ps1 -SoarResourceGroup rg-sentinel-soar -Days 7 Requires modules: Az.Accounts, Az.LogicApp. EXPECTED RESULTS A per-playbook table of run counts by status over the window, and a highlighted list of any 'Response - Contain - Pilot' runs for manual review. Use run counts as a Logic App cost proxy. .NOTES Sentinel Solutions. Read-only. > [CmdletBinding()] param( [string]$SoarResourceGroup = 'rg-sentinel-soar', [int]$Days = 7, [string]$ResponsePlaybook = 'Response - Contain - Pilot' )

## SENTSOAR-Set-PlaybookIdentity.ps1

**Purpose:** Reports each playbook's system-assigned managed identity and its role assignments so you can confirm least privilege (Microsoft Sentinel Responder for incident updates; NO enforced-action role on the response playbook during the pilot). Can OPTIONALLY assign Microsoft Sentinel Responder to a playbook's identity — but defaults to -WhatIf (reports only, changes nothing). HOW TO RUN Connect-AzAccount first. Report only (default, safe): powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Set-PlaybookIdentity.ps1 -SoarResourceGroup rg-sentinel-soar -WorkspaceResourceId <workspace-resource-id> Assign Responder to one playbook's identity (must pass -WhatIf:$false): powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Set-PlaybookIdentity.ps1 -SoarResourceGroup rg-sentinel-soar -WorkspaceResourceId <id> -AssignResponderTo "Enrichment - Incident - Pilot" -WhatIf:$false Requires modules: Az.Accounts, Az.LogicApp, Az.Resources. EXPECTED RESULTS Per-playbook identity + role list. In preview: prints the role it WOULD assign. When applied: grants Microsoft Sentinel Responder to the named playbook's identity at the workspace scope. .NOTES Sentinel Solutions. Mutating only when -AssignResponderTo is set AND -WhatIf:$false is passed. > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param( [string]$SoarResourceGroup = 'rg-sentinel-soar', [Parameter(Mandatory)] [string]$WorkspaceResourceId, [string]$AssignResponderTo, [bool]$WhatIf = $true )

**How to run:** Connect-AzAccount first. Report only (default, safe): powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Set-PlaybookIdentity.ps1 -SoarResourceGroup rg-sentinel-soar -WorkspaceResourceId <workspace-resource-id> Assign Responder to one playbook's identity (must pass -WhatIf:$false): powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Set-PlaybookIdentity.ps1 -SoarResourceGroup rg-sentinel-soar -WorkspaceResourceId <id> -AssignResponderTo "Enrichment - Incident - Pilot" -WhatIf:$false Requires modules: Az.Accounts, Az.LogicApp, Az.Resources. EXPECTED RESULTS Per-playbook identity + role list. In preview: prints the role it WOULD assign. When applied: grants Microsoft Sentinel Responder to the named playbook's identity at the workspace scope. .NOTES Sentinel Solutions. Mutating only when -AssignResponderTo is set AND -WhatIf:$false is passed. > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param( [string]$SoarResourceGroup = 'rg-sentinel-soar', [Parameter(Mandatory)] [string]$WorkspaceResourceId, [string]$AssignResponderTo, [bool]$WhatIf = $true )

## SENTSOAR-SimulateResponse.ps1

**Purpose:** Mirrors what the 'Response - Contain - Pilot' playbook would do, but as a SAFE, read-only dry run: given an incident's entities (a user UPN, a device id, an IP), it prints exactly which disable-user / isolate-device / block-IP action WOULD run, checks the target exists, and makes NO change. Use it to rehearse containment and to demonstrate simulate-first behaviour. HOW TO RUN Connect-AzAccount (and Connect-MgGraph for the user check) first, then: powershell -ExecutionPolicy Bypass -File .\SENTSOAR-SimulateResponse.ps1 -UserUpn user@contoso.com -DeviceId <mde-machine-id> -IpAddress 203.0.113.10 Any parameter may be omitted; only the provided targets are simulated. Requires modules: Az.Accounts, Microsoft.Graph.Users (for the optional user lookup). EXPECTED RESULTS For each provided target: a "WOULD ..." line describing the enforced action, a lookup result confirming the target resolves, and an explicit "NO CHANGE MADE (simulate)" note. Never mutates. .NOTES Sentinel Solutions. Read-only / simulate-only by design — this script contains no enforcing calls. > [CmdletBinding()] param( [string]$UserUpn, [string]$DeviceId, [string]$IpAddress )

**How to run:** Connect-AzAccount (and Connect-MgGraph for the user check) first, then: powershell -ExecutionPolicy Bypass -File .\SENTSOAR-SimulateResponse.ps1 -UserUpn user@contoso.com -DeviceId <mde-machine-id> -IpAddress 203.0.113.10 Any parameter may be omitted; only the provided targets are simulated. Requires modules: Az.Accounts, Microsoft.Graph.Users (for the optional user

lookup). EXPECTED RESULTS For each provided target: a "WOULD ..." line describing the enforced action, a lookup result confirming the target resolves, and an explicit "NO CHANGE MADE (simulate)" note. Never mutates. .NOTES Sentinel Solutions. Read-only / simulate-only by design — this script contains no enforcing calls. > [CmdletBinding()] param( [string]$UserUpn, [string]$DeviceId, [string]$IpAddress )

## SENTSOAR-Validate.ps1

**Purpose:** Confirms the pilot automation rule is enabled, each expected playbook exists, is enabled, and has a system-assigned managed identity, and reports whether the response playbook still looks simulate-first (no enforced-action platform role assigned). Read-only — no changes. HOW TO RUN Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Validate.ps1 -ResourceGroup <workspace-rg> -WorkspaceName "..." -SoarResourceGroup rg-sentinel-soar Requires modules: Az.Accounts, Az.SecurityInsights, Az.LogicApp, Az.Resources. EXPECTED RESULTS PASS/CHECK lines for: automation rule 'Auto-Triage - New Incidents - Pilot' enabled; the four playbooks present and enabled; each playbook has a managed identity. Highlights anything off. .NOTES Sentinel Solutions. Read-only. > [CmdletBinding()] param( [Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [string]$SoarResourceGroup = 'rg-sentinel-soar', [string]$AutomationRuleName = 'Auto-Triage - New Incidents - Pilot', [string[]]$Playbooks = @('Enrichment - Incident - Pilot','Notify - Teams-Email - Pilot','Ticketing - Incident - Pilot','Response - Contain - Pilot') )

**How to run:** Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SENTSOAR-Validate.ps1 -ResourceGroup <workspace-rg> -WorkspaceName "..." -SoarResourceGroup rg-sentinel-soar Requires modules: Az.Accounts, Az.SecurityInsights, Az.LogicApp, Az.Resources. EXPECTED RESULTS PASS/CHECK lines for: automation rule 'Auto-Triage - New Incidents - Pilot' enabled; the four playbooks present and enabled; each playbook has a managed identity. Highlights anything off. .NOTES Sentinel Solutions. Read-only. > [CmdletBinding()] param( [Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [string]$SoarResourceGroup = 'rg-sentinel-soar', [string]$AutomationRuleName = 'Auto-Triage - New Incidents - Pilot', [string[]]$Playbooks = @('Enrichment - Incident - Pilot','Notify - Teams-Email - Pilot','Ticketing - Incident - Pilot','Response - Contain - Pilot') )